

Python CSV Quick & Simple Guide | Read, Write & Manipulate (Updated 2024)

[BEGINNER](#)[LISTICLE](#)[PANDAS](#)[PROGRAMMING](#)[PYTHON](#)[STRUCTURED DATA](#)[TECHNIQUE](#)

Introduction

Pandas csv tutorial is a typical file format that is often used in the industry to store data because it stores data in a typical text file and maintains a tabular format. Thus, it is necessary to have a good understanding of the CSV format to easily handle the data you are going to deal with on a day-to-day basis. In this beginner-friendly article, I'll introduce you to the Pandas CSV Operations format and show you how to handle **CSV format in python**.

Learning Objectives

- In this tutorial, you will learn about the CSV file format and its basic operations using Python.
- We will create a Pandas DataFrame by importing a local csv file and importing csv file from a URL.
- Lastly, we will see **how to handle CSV** format under different circumstances.

This article was published as a part of the [Data Science Blogathon](#).

Table of contents

- [Introduction](#)
- [What Is a CSV?](#)
- [Basic Operations With CSV Files](#)
- [Why Do We Use CSV Files?](#)
- [How to Read CSV File using Pandas read_csv](#)
- [Pandas DataFrame](#)
- [Different Parameters in CSV](#)
- [Conclusion](#)
- [Frequently Asked Questions](#)

What Is a CSV?

CSV (Comma Separated Values) may be a simple file format accustomed to storing tabular data, like a spreadsheet or [database](#). CSV file stores tabular data (numbers and text) in plain text. Each line of the file could be a data record. Each record consists of 1 or more fields, separated by commas. The use of the comma as a field separator is the source of the name for this file format.

Basic Operations With CSV Files

In Basic operations, we are going to understand the subsequently three things:

1. Working with CSV files
2. Opening a CSV file
3. Saving a CSV file

How to Work With CSV Files?

Working with CSV files isn't a tedious task, but it's pretty straightforward. However, counting on your workflow, there can become caveats that you might want to observe out for.

How to Open a CSV File?

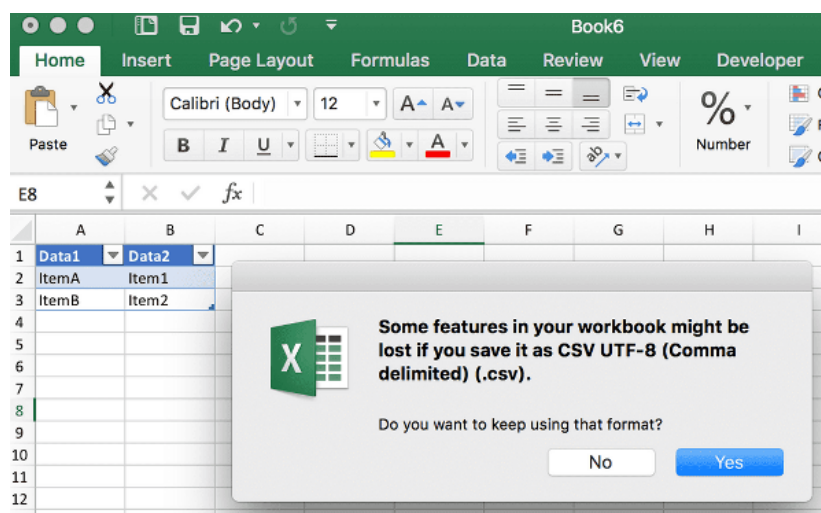
If you've got a [CSV file](#), you'll open it in Excel without much trouble. Just open Excel, and find the CSV file to work with (or right-click on the CSV file and choose Open in Excel). After you open the file, you'll notice that the data is simply plain text put into different cells. When dealing with CSV files in Python using pandas, you can perform various operations like reading, writing, and manipulating the data easily.

How to Save a CSV File?

If you wish to save lots of your current workbook into a CSV file, you have got to use the subsequent commands:

File -> Save As... and choose CSV file.

More often than not, you'll get this warning:



Let's understand what this error is saying to us.

Here Excel is trying to mention that your CSV Files don't save any reasonable formatting in the least.

For Example, Column widths, font styles, colors, etc., won't be saved.

Just your plain old data are saved in an exceedingly comma-separated file.

Note that even after you put it aside, Excel will still show the formats you just had, so don't be fooled by this and think that after you open the workbook again, your formats will still be there. They won't be.

Even after you open up a CSV get in Excel, if you apply even any minor formatting, like adjusting the column widths to work out the info, Excel will still warn you that you can't save the formats that you just added. You'll get a warning like this one:

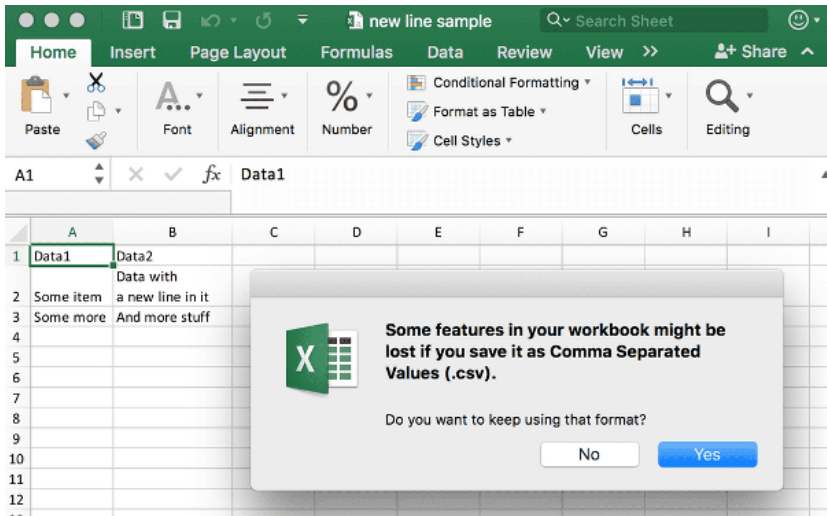


Image Source: Google Images

So, the aim is to note that your formats can never be saved in CSV Files.

Why Do We Use CSV Files?

CSV files are the simplest way to speak data between different applications, making them versatile for data exchange. Say you had a database application and wanted to export the info to a file. If you wish to export it to an Excel file, the database application will support exporting to XLS* files. Additionally, when working with CSV files in Python, particularly with pandas CSV operations, you can easily read, write, and manipulate tabular data for analysis and processing.

However, since the [CSV file](#) format is extremely straightforward and lightweight (much more than XLS* files), it's easier for varied applications to support it. In its basic usage, you have a line of text, with every column of data going in alternative ways by a comma. That's it. And since of this simplicity, it's simple for developers to make Export / Import practicality with CSV files to transfer knowledge between applications instead of a lot of sophisticated file formats.

For Example,

Let's take the **tabular data** in the given form below:

Table Data			
Programming language	Designed by	Appeared	Extension
Python	Guido van Rossum	1991	.py
Java	James Gosling	1995	.java
C++	Bjarne Stroustrup	1983	.cpp

If we convert this data into a **CSV Format**, then it looks like this:

CSV Data

Programming language, Designed by, Appeared, Extension

Python, Guido van Rossum, 1991, .py

Java, James Gosling, 1995, .java

C++, Bjarne Stroustrup, 1983, .cpp

Now, we are completed with all the basics of CSV files. So, In the later part of the article, we will be discussing working with CSV files in a detailed manner.

How to Read CSV File using Pandas read_csv

To read a CSV file using Pandas' read_csv function, you first need to import the Pandas library. Then, you can use the read_csv function by passing the path to the CSV file as an argument. This function will return a DataFrame containing the data from the CSV file. Here's an example [code snippet](#):

```
import pandas as pd # Path to the CSV file csv_file_path = "path/to/your/file.csv" # Read the CSV file into a DataFrame df = pd.read_csv(csv_file_path) # Now you can work with the DataFrame 'df'
```

Replace "path/to/your/file.csv" with the actual path to your CSV file. Once the CSV file is read into the DataFrame, you can perform various operations and analysis on the data using Pandas.

Pandas DataFrame

One of the easiest ways to read a csv file in Python is using the Pandas library. It has a very useful function, read_csv(), which allows us to read a CSV file and create a Pandas DataFrame. A Pandas DataFrame is nothing but a two-dimensional data structure storing data like a table with rows and columns. So here I'll show you how to work with CSV files using Python Pandas.

Importing Pandas

Firstly, we import the necessary the **Pandas** Library of Python.

```
import pandas as pd
```

Now we will see how to load and read the dataset easily.

Read-csv Function

- It is an important pandas function to read CSV files and do operations on them.
- This function helps us load the file from your local machine or any URL.

Opening a Local CSV File

If the file is present in the same location as in our Python File, then give the file name only to load that file; otherwise, you have to give the complete filepath to the file. Following is the syntax to read a csv file and create a pandas dataframe from it.

```
df = pd.read_csv('aug_train.csv') df
```

Output:

	enrollee id	city	city development index	gender	relevant experience	enrolled university	education level	major discipline	experience	company s
0	8540	city_103	0.620	Male	Has relevant experience	no_enrollment	Graduate	STEM	>20	h
1	28726	city_20	0.776	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50
2	11561	city_21	0.624	NaN	No relevant experience	Full time course	Graduate	STEM	5	h
3	35241	city_115	0.789	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	h
4	666	city_182	0.767	Male	Has relevant experience	no_enrollment	Masters	STEM	>20	50
...	...	...	...	...	...	...	...	...	...	...
19153	7386	city_173	0.678	Male	No relevant experience	no_enrollment	Graduate	Humanities	14	h
19154	31386	city_103	0.620	Male	Has relevant experience	no_enrollment	Graduate	STEM	14	h
19155	24576	city_103	0.620	Male	Has relevant experience	no_enrollment	Graduate	STEM	>20	50
19156	5756	city_85	0.802	Male	Has relevant experience	no_enrollment	High School	NaN	<1	5004
19157	22824	city_57	0.860	NaN	No relevant experience	no_enrollment	Primary School	NaN	2	h

19158 rows x 14 columns

Opening a CSV File From a URL

If the file is not present directly in our local machine, but we have to fetch the data from a given URL, then we take the help of the requests module to load that data.

Python Code:

main.py

```
1 #Import Library
2 from sklearn import svm
3 import pandas as pd
4 from sklearn.metrics import accuracy_score
5 import warnings
6 warnings.filterwarnings("ignore")
7
8 #Load Train and Test datasets
9 #Identify the features and target variable
10 train=pd.read_csv('train_data.csv')
11 train=train[['Loan_Status','Loan_Amount_Term']]
12 train=train.drop(['Loan_Status'],axis=1)
```

Login/ Signup to View & Run Code in the browser

View & Run Code

Output:

	Country	Region
0	Algeria	AFRICA
1	Angola	AFRICA
2	Benin	AFRICA
3	Botswana	AFRICA
4	Burkina	AFRICA
...	...	...
189	Paraguay	SOUTH AMERICA
190	Peru	SOUTH AMERICA
191	Suriname	SOUTH AMERICA
192	Uruguay	SOUTH AMERICA
193	Venezuela	SOUTH AMERICA

194 rows × 2 columns

Different Parameters in CSV

Sep Parameter

Suppose we have a dataset in which entities in a particular row are not separated by a comma but by some other delimiter or separator. In that case, we have to use the sep parameter to specify that specific separator or delimiter. For example, a tab or a semi-colon could also be used as separtors.

Say, we have a tsv file, i.e., entities are tab-separated, and if we try to load this data directly, then loads of all the entities are combined.

```
import pandas as pd
pd.read_csv('movie_titles_metadata.tsv')
```

Output:

	m0\t10 things i hate about you\t1999\t6.90\t62847\t['comedy' 'romance']
0	m1\t1492: conquest of paradise\t1992\t6.20\t10...
1	m2\t15 minutes\t2001\t6.10\t25854\t['action' '...
2	m3\t2001: a space odyssey\t1968\t8.40\t163227\...
3	m4\t48 hrs.\t1982\t6.90\t22289\t['action' 'com...
4	m5\tthe fifth element\t1997\t7.50\t133756\t['a...
...	...
611	m612\twatchmen\t2009\t7.80\t135229\t['action' ...
612	m613\txxx\t2002\t5.60\t53505\t['action' 'adven...
613	m614\tx-men\t2000\t7.40\t122149\t['action' 'sc...
614	m615\tyoung frankenstein\t1974\t8.00\t57618\t[...
615	m616\tzulu dawn\t1979\t6.40\t19111\t['action' '...

616 rows × 1 columns

To solve the above problem for the CSV file, we have to overwrite the sep parameter to ‘t’ instead of ‘;’ which is a default separator.

```
import pandas as pd
pd.read_csv('movie_titles_metadata.tsv',sep='t')
```

Output:

	m0	10 things i hate about you	1999	6.90	62847	['comedy' 'romance']
0	m1	1492: conquest of paradise	1992	6.2	10421.0	['adventure' 'biography' 'drama' 'history']
1	m2	15 minutes	2001	6.1	25854.0	['action' 'crime' 'drama' 'thriller']
2	m3	2001: a space odyssey	1968	8.4	163227.0	['adventure' 'mystery' 'sci-fi']
3	m4	48 hrs.	1982	6.9	22289.0	['action' 'comedy' 'crime' 'drama' 'thriller']
4	m5	the fifth element	1997	7.5	133756.0	['action' 'adventure' 'romance' 'sci-fi' 'thri...
...	...	...	...	...	...	...
611	m612	watchmen	2009	7.8	135229.0	['action' 'crime' 'fantasy' 'mystery' 'sci-fi']...
612	m613	xxx	2002	5.6	53505.0	['action' 'adventure' 'crime']
613	m614	x-men	2000	7.4	122149.0	['action' 'sci-fi']
614	m615	young frankenstein	1974	8.0	57618.0	['comedy' 'sci-fi']
615	m616	zulu dawn	1979	6.4	1911.0	['action' 'adventure' 'drama' 'history' 'war']

616 rows x 6 columns

In the above example, we have observed that the first row is treated as the column’s name, and to solve this problem and make our custom name for the columns, we have to specify the list of words with names as the name of the list.

```
pd.read_csv('movie_titles_metadata.tsv',sep='t',names=
['sno','name','release_year','rating','votes','genres'])
```

Output:

	sno	name	release_year	rating	votes	genres
0	m0	10 things i hate about you	1999	6.9	62847.0	['comedy' 'romance']
1	m1	1492: conquest of paradise	1992	6.2	10421.0	['adventure' 'biography' 'drama' 'history']
2	m2	15 minutes	2001	6.1	25854.0	['action' 'crime' 'drama' 'thriller']
3	m3	2001: a space odyssey	1968	8.4	163227.0	['adventure' 'mystery' 'sci-fi']
4	m4	48 hrs.	1982	6.9	22289.0	['action' 'comedy' 'crime' 'drama' 'thriller']
...	...	...	...	...	...	...
612	m612	watchmen	2009	7.8	135229.0	['action' 'crime' 'fantasy' 'mystery' 'sci-fi']...
613	m613	xxx	2002	5.6	53505.0	['action' 'adventure' 'crime']
614	m614	x-men	2000	7.4	122149.0	['action' 'sci-fi']
615	m615	young frankenstein	1974	8.0	57618.0	['comedy' 'sci-fi']
616	m616	zulu dawn	1979	6.4	1911.0	['action' 'adventure' 'drama' 'history' 'war']

617 rows x 6 columns

Index-col Parameter

The index-col parameter allows us to set which columns to use as the data frame’s index. The default value for this parameter is None, and pandas automatically will add a new column starting from 0 to describe the index column.

So, it allows us to use a column as the row labels for a given DataFrame. This function comes in handy quite often. Let's say we have an ID column present with our dataset, which is not impacting our predictions, so we make that column our index for rows instead of the default.

```
pd.read_csv('aug_train.csv', index_col='enrollee_id')
```

Output:

	city	city_development_index	gender	relevant_experience	enrolled_university	education_level	major_discipline	experience	company_size	con
enrollee_id										
8949	city_103	0.920	Male	Has relevant experience	no_enrollment	Graduate	STEM	>50	NaN	
29725	city_40	0.778	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50-99	
11561	city_21	0.624	NaN	No relevant experience	Full time course	Graduate	STEM	5	NaN	
33241	city_115	0.789	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	NaN	
666	city_162	0.787	Male	Has relevant experience	no_enrollment	Masters	STEM	>20	50-99	
...	...	...	...	...	...	...	...	...	...	
7396	city_173	0.878	Male	No relevant experience	no_enrollment	Graduate	Humanities	14	NaN	
31598	city_103	0.920	Male	Has relevant experience	no_enrollment	Graduate	STEM	14	NaN	
24576	city_103	0.920	Male	Has relevant experience	no_enrollment	Graduate	STEM	>50	50-99	
5756	city_85	0.802	Male	Has relevant experience	no_enrollment	High School	NaN	<1	500-999	
23634	city_57	0.855	NaN	No relevant experience	no_enrollment	Primary School	NaN	2	NaN	

19158 rows x 11 columns

Header Parameter

Header parameter allows us to specify which row will be used as column names for your data frame. It expects input as an int value or a list of int values.

The default value for this parameter is **header=0**, which implies that the first row of the CSV file will be considered as column names.

```
pd.read_csv('test.csv', header=1)
```

Output:

	enrollee_id	city	city_development_index	gender	relevant_experience	enrolled_university	education_level	major_discipline	experience	company_size	
0	1	28725	city_40	0.778	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50-99
1	2	11561	city_21	0.624	NaN	No relevant experience	Full time course	Graduate	STEM	5	NaN
2	3	33241	city_115	0.789	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	NaN
3	4	666	city_162	0.787	Male	Has relevant experience	no_enrollment	Masters	STEM	>20	50-99

Use-cols Parameter

The use-cols parameter specifies which columns to import from the complete dataset to the data frame. It can take input from either a list of int values or directly from the column names.

This function comes in handy when we have to analyze data on just some columns and not on all the columns of our dataset.

So, this parameter returns a subset of the columns from your dataset.


```
pd.read_csv('aug_train.csv',usecols=['enrollee_id','gender','education_level'])
```

Output:

	enrollee_id	gender	education_level
0	8949	Male	Graduate
1	29725	Male	Graduate
2	11561	NaN	Graduate
3	33241	NaN	Graduate
4	666	Male	Masters
...	...	...	...
19153	7386	Male	Graduate
19154	31398	Male	Graduate
19155	24576	Male	Graduate
19156	5756	Male	High School
19157	23834	NaN	Primary School

19158 rows × 3 columns

Squeeze Parameter

If true and only one column is passed, the squeeze parameter returns a pandas series instead of a DataFrame. A Pandas series is a single-dimensional data structure storing only a single column with a single data type.

```
pd.read_csv('aug_train.csv',usecols=['gender'],squeeze=True)
```

Output:

```
0      Male
1      Male
2      NaN
3      NaN
4      Male
...
19153  Male
19154  Male
19155  Male
19156  Male
19157  NaN
Name: gender, Length: 19158, dtype: object
```

Skiprows Parameter

Skiprows parameter is used to skip past rows in the new data frame. This is useful as we can easily iterate through the csv data and read only the relevant rows.

```
pd.read_csv('aug_train.csv',skiprows=[0,1])
```

Output:

	29725	city_40	0.7759999999999999	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50-99	Pvt Ltd	>4	47	0.0
0	11061	city_21	0.624	NaN	No relevant experience	Full time course	Graduate	STEM	5	NaN	NaN	never	88	0.0
1	88241	city_115	0.780	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	NaN	Pvt Ltd	never	59	1.0
2	698	city_182	0.767	Male	Has relevant experience	no_enrollment	Masters	STEM	>20	50-99	Funded Startup	4	8	0.0
3	21651	city_176	0.764	NaN	Has relevant experience	Part time course	Graduate	STEM	11	NaN	NaN	1	24	1.0
4	28805	city_190	0.920	Male	Has relevant experience	no_enrollment	High School	NaN	5	50-99	Funded Startup	1	24	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19151	7396	city_173	0.878	Male	No relevant experience	no_enrollment	Graduate	Humanities	14	NaN	NaN	1	42	1.0
19162	51598	city_103	0.920	Male	Has relevant experience	no_enrollment	Graduate	STEM	14	NaN	NaN	4	52	1.0
19153	54676	city_103	0.920	Male	Has relevant experience	no_enrollment	Graduate	STEM	>20	90-99	Pvt Ltd	4	44	0.0
19154	5758	city_65	0.802	Male	Has relevant experience	no_enrollment	High School	NaN	<1	500-999	Pvt Ltd	2	87	0.0
19155	28834	city_57	0.850	NaN	No relevant experience	no_enrollment	Primary School	NaN	2	NaN	NaN	1	127	0.0

19156 rows x 14 columns

Nrows Parameter

The nrows function only reads the fixed number of rows from the file. Needs an int value.

This parameter comes in handy when we have a huge dataset, and we want to load our dataset in chunks instead of directly loading the complete dataset.

```
pd.read_csv('aug_train.csv',nrows=100)
```

Output:

	enrollee_id	city	city_development_index	gender	relevant_experience	enrolled_university	education_level	major_discipline	experience	company_size
0	8948	city_103	0.950	Male	I has relevant experience	no_enrollment	Graduate	STEM	>20	NaN
1	28725	city_50	0.776	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50-99
2	11561	city_21	0.624	NaN	No relevant experience	Full time course	Graduate	STEM	5	NaN
3	33241	city_115	0.780	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	NaN
4	666	city_182	0.767	Male	Has relevant experience	no_enrollment	Masters	STEM	>20	50-99
...	...	...	...	...	...	...	...	...	...	...
95	13281	city_50	0.802	Male	I has relevant experience	Full time course	Graduate	STEM	8	100-99
96	7364	city_160	0.820	NaN	No relevant experience	Full time course	High School	NaN	2	100-500
97	11184	city_74	0.570	NaN	No relevant experience	Full time course	Graduate	STEM	2	100-500
98	7016	city_85	0.802	Male	Has relevant experience	no_enrollment	Graduate	STEM	8	50-99
99	8895	city_11	0.850	Male	I has relevant experience	no_enrollment	Graduate	STEM	8	10149

100 rows x 11 columns

Encoding Parameter

The encoding parameter helps determine which encoding you must use for UTF when reading or writing the files.

Sometimes, our files are not encoded properly, i.e., UTF-8. So, saving that with a text editor or adding the param “**encoding='utf-8'**” doesn’t work. In both cases, it returns the error.

So, to resolve this issue, we call our read_csv function with **encoding='latin1'**, **encoding='iso-8859-1'** or **encoding='cp1252'** (these are some of the various encodings found on Windows).

```
pd.read_csv('zomato.csv',encoding='latin-1')
```

Output:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose	Longitude	Latitude	Cuisines	...	Currency	Has Table booking	Is Online deliv
0	6317837	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenue...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...	-21.027535	14.585443	French, Japanese, Desserts	...	Philippine	Yes	
1	6304297	Izakaya Kizufu	162	Makati City	Little Tokyo, 2277 Chino Flores Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...	-21.014101	14.553706	Japanese	...	Philippine	Yes	
2	6300032	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Borden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...	-21.059931	14.581404	Seafood, Asian, Filipino, Indian	...	Philippine	Yes	
3	6318538	Oorra	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, G...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	-21.059475	14.585316	Japanese, Sushi	...	Philippine	No	
4	6314332	Semco Kafe	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	-21.057508	14.584450	Japanese, Korean	...	Philippine	Yes	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9546	6916730	NamiDa Bunsu	208	Üstanbul	Kemankab Karamustafa Pazarı, Kulevesi	Karaköy	Karaköy, Üsküdar	28.977392	41.022798	Turkish	...	Turkish Lira(TRY)	No	

Error-bad-lines Parameter

If we have a dataset in which some lines is having too many fields (**For Example**, a CSV line with too many commas), then by default, it raises and causes an exception, and no DataFrame will be returned.

So, to resolve these types of issues, we have to make this parameter False, then these “bad lines” will be dropped from the DataFrame that is returned. (**Only valid with C parser**)

```
pd.read_csv('BX-Books.csv', sep=';', encoding="latin-1",error_bad_lines=False)
```

Output:

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	Image-URL-S
0	0186153449	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.com/images/P/0186153448.D... http://images.amazon.com/images
1	0002003018	Clara Callan	Richard Bruce Wright	2001	HarperCollins Canada	http://images.amazon.com/images/P/0002003018.D... http://images.amazon.com/images
2	0000943129	Decision in Normandy	Carlo D'Este	1991	HarperPerennial	http://images.amazon.com/images/P/0000943129.D... http://images.amazon.com/images
3	0374157095	Flu: The Story of the Great Influenza Pandemic...	Gina Bari Kosloski	1999	Farrar Straus Giroux	http://images.amazon.com/images/P/0374157095.D... http://images.amazon.com/images
4	0383045218	The Mummies of Urumchi	F. J. W. Barber	1999	W. W. Norton & Company	http://images.amazon.com/images/P/0383045218.D... http://images.amazon.com/images
...	...	...	...	...	...	...
271355	0440400999	There's a Bat in Burk Five	Paula Danziger	1988	Random House Children's Pub. (Inf)	http://images.amazon.com/images/P/0440400999.D... http://images.amazon.com/images
271356	0026447944	From One to One Hundred	Ian Spar	1991	Dutton Books	http://images.amazon.com/images/P/0026447944.D... http://images.amazon.com/images
271367	008008667X	City Data: The Inc Story of the Town that Ta...	Christina Wiener	2004	HarperSanFrancisco	http://images.amazon.com/images/P/008008667X.D... http://images.amazon.com/images
271358	0182120040	Republic (World's Classics)	Plato	1996	Oxford University Press	http://images.amazon.com/images/P/0182120040.D... http://images.amazon.com/images
		A Guided Tour of				

Dtype Parameter

The dtype parameter stands for 'data type' for converting data or columns. **For Example**, {'a': np.float64, 'b': np.int32}

This function comes in handy when we need to convert our columns from float data type to int data type.

```
pd.read_csv('aug_train.csv',dtype={'target':int}).info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19158 entries, 0 to 19157
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   enrollee_id                          19158 non-null  int64
1   city                                  19158 non-null  object
2   city_development_index               19158 non-null  float64
3   gender                               14650 non-null  object
4   relevent_experience                  19158 non-null  object
5   enrolled_university                 18772 non-null  object
6   education_level                     18698 non-null  object
7   major_discipline                    16345 non-null  object
8   experience                           19093 non-null  object
9   company_size                        13220 non-null  object
10  company_type                         13018 non-null  object
11  last_new_job                         18735 non-null  object
12  training_hours                      19158 non-null  int64
13  target                              19158 non-null  int32
dtypes: float64(1), int32(1), int64(2), object(10)
memory usage: 2.0+ MB
```

Parse-dates Parameter

If we make this parameter True, then it tries to parse the index.

For Example, If [1, 2, 3] -> try parsing columns 1, 2, 3 each as a separate date column, and if we have to combine columns 1 and 3 and parse them as a single date column, then use [[1,3]].

```
pd.read_csv('IPL Matches 2008-2020.csv',parse_dates=['date']).info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 816 entries, 0 to 815
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype
---  -
0   id                   816 non-null    int64
1   city                 803 non-null    object
2   date                 816 non-null    datetime64[ns]
3   player_of_match      812 non-null    object
4   venue                816 non-null    object
5   neutral_venue        816 non-null    int64
6   team1                816 non-null    object
7   team2                816 non-null    object
8   toss_winner          816 non-null    object
9   toss_decision        816 non-null    object
10  winner               812 non-null    object
11  result               812 non-null    object
12  result_margin        799 non-null    float64
13  eliminator           812 non-null    object
14  method               19 non-null     object
15  umpire1              816 non-null    object
16  umpire2              816 non-null    object
dtypes: datetime64[ns](1), float64(1), int64(2), object(13)
memory usage: 108.5+ KB
```

Convertors Parameter

This parameter helps us to convert values in the columns based on a custom function given by the user.

```
def rename(name): if name == "Royal Challengers Bangalore": return "RCB" else: return name
```

```
rename("Royal Challengers Bangalore")
```

Output:

‘RCB’

```
pd.read_csv('IPL Matches 2008-2020.csv',converters={'team1':rename})
```

Output:

	id	city	date	player_of_match	venue	neutral_venue	team1	team2	toss_winner	toss_decision	winner	result	result
0	886982	Bangalore	2009-04-18	BB McCullum	Chinnaswamy Stadium	0	RCB	Kolkata Knight Riders	Royal Challengers Bangalore	field	Kolkata Knight Riders	runs	
1	886983	Chandigarh	2009-04-19	MEK Hussey	Punjab Cricket Association Stadium, Mohali	0	Kings XI Punjab	Chennai Super Kings	Chennai Super Kings	bat	Chennai Super Kings	runs	
2	886984	Delhi	2009-04-19	MF Maharoof	Feroz Shah Kotla	0	Delhi Daredevils	Rajasthan Royals	Rajasthan Royals	bat	Delhi Daredevils	wickets	
3	886985	Mumbai	2009-04-20	MV Boucher	Wankhede Stadium	0	Mumbai Indians	Royal Challengers Bangalore	Mumbai Indians	bat	Royal Challengers Bangalore	wickets	
4	886986	Kolkata	2009-04-20	DJ Hussey	Eden Gardens	0	Kolkata Knight Riders	Deccan Chargers	Deccan Chargers	bat	Kolkata Knight Riders	wickets	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
811	1216547	Dubai	2020-09-28	AB de Villiers	Dubai International Cricket Stadium	0	RCB	Mumbai Indians	Mumbai Indians	field	Royal Challengers Bangalore	tie	
812	1257177	Dubai	2020-11-05	JJ Bawaish	Dubai International Cricket Stadium	0	Mumbai Indians	Dalhi Capitals	Dalhi Capitals	field	Mumbai Indians	runs	
813	1257178	Abu Dhabi	2020-11-06	KS Williamson	Sheikh Zayed Stadium	0	RCB	Sunrisers Hyderabad	Sunrisers Hyderabad	field	Sunrisers Hyderabad	wickets	
814	1257180	Abu Dhabi	2020-11-06	MP Stoinis	Sheikh Zayed Stadium	0	Dalhi Capitals	Sunrisers Hyderabad	Dalhi Capitals	bat	Dalhi Capitals	runs	

Na-values Parameter

As we know, the missing values by default will be NaN. If we want other strings to be considered as NaN, then we have to use this parameter. It expects a list of strings as the input.

Sometimes in our dataset, another type of symbol is used to make them missing values, so at that time to understand those values as missing, we use this parameter.

```
pd.read_csv('aug_train.csv',na_values=['Male',])
```

Output:

	enrollae id	city	city development index	gender	relevant experience	enrolled university	education level	major discipline	experience	company s
0	8549	city_103	0.390	Male	Has relevant experience	no_enrollment	Graduate	STEM	>20	N
1	25725	city_40	0.776	Male	No relevant experience	no_enrollment	Graduate	STEM	15	50
2	11561	city_21	0.824	NaN	No relevant experience	Full time course	Graduate	STEM	5	N
3	33541	city_115	0.789	NaN	No relevant experience	NaN	Graduate	Business Degree	<1	N
4	666	city_169	0.767	Male	Has relevant experience	no_enrollment	Master's	STEM	>20	50
...	...	...	...	...	...	...	...	...	...	...
19153	7286	city_173	0.878	Male	No relevant experience	no_enrollment	Graduate	Humanities	14	N
19184	81596	city_103	0.390	Male	Has relevant experience	no_enrollment	Graduate	STEM	14	N
19155	24578	city_103	0.820	Male	Has relevant experience	no_enrollment	Graduate	STEM	>20	50
19156	5166	city_85	0.802	Male	Has relevant experience	no_enrollment	High School	NaN	<1	100+
19187	28854	city_67	0.855	NaN	No relevant experience	no_enrollment	Primary School	NaN	2	N

19158 rows x 11 columns

This completes our discussion!

NOTE: In this article, we will only discuss those very useful parameters while working with CSV files daily. But if you are interested to learn more parameters, then refer to the official website of Pandas.

Conclusion

So in this beginner-friendly tutorial, we gained a good understanding of the [Pandas CSV](#) Operations format and how to handle it in Python. We looked at how to create a Pandas DataFrame and Pandas read CSV file using read_csv() function. We also looked at various parameters of read_csv() function like sep, header, nrows, etc. parameters to read the CSV file in Pandas in the most efficient manner to serve our purpose.

Key Takeaways

- CSV format is a simple file format that stores data in a tabular form, like a spreadsheet or database.
- CSV format is simple, straightforward, lightweight, and supported by various applications.

Frequently Asked Questions

Q1.What is csv operations?

A. CSV operations, like reading, writing, and manipulating data, are essential parts of working with CSV files. This is covered in a Pandas CSV tutorial.

Q2. What are the advantages of reading a CSV file in Pandas?

A. In Pandas, we read a CSV file using the read_csv() function. This function supports a lot of parameters which makes reading CSV files very easy. For example, we can easily read a large CSV file in chunks using the nrows parameters, which read only a set number of rows. Or the skiprows parameter, which allows skipping a certain number of rows while reading a CSV file.

Q3. How do pandas work with CSV files in python?

A. read_csv() function imports a CSV file to DataFrame format.

Q4.How to manipulate csv with Pandas?

To work with CSV files using Pandas, follow a Pandas CSV tutorial. You can read, write, and manipulate data easily once you've imported the Pandas library in Python.

Q5.How to iterate through csv using Pandas?

Learn how to iterate through CSV files with Pandas in a simple Pandas CSV tutorial. After reading the file into a DataFrame, you can access and manipulate each row of data.

Article Url - <https://www.analyticsvidhya.com/blog/2021/06/complete-guide-to-working-with-csv-files-in-python-with-pandas/>



CHIRAG GOYAL

I am currently pursuing my Bachelor of Technology (B.Tech) in Computer Science and Engineering from the Indian Institute of Technology Jodhpur(IITJ). I am very enthusiastic about Machine learning, Deep Learning, and Artificial Intelligence. Feel free to connect with me on LinkedIn.